

SlackHelper Class

Apex Code

```
public with sharing class SlackHelper {
    public class ReturnToken {
        public String token;
        public String message;
    }

    public class ReturnSlackCallout {
        public string status;
        public String message;
        public HttpResponse res;
    }

    public static ReturnToken getBotToken(String agentName, String userSlackBotToken)
    {
        ReturnToken result = new ReturnToken();

        SlackBotToken__mdt tokenMetadata = [
            SELECT Bot_User_OAuth_Token__c
            FROM SlackBotToken__mdt
            WHERE DeveloperName = :agentName
            LIMIT 1
        ];

        // Use the token from metadata if available
        if (tokenMetadata != null) {
            result.token = tokenMetadata.Bot_User_OAuth_Token__c;
            result.message = 'No token provided, using token from custom metadata';
        } else {
            result.token = null;
        }

        // Override with the user-provided token if available
        if (!String.isBlank(userSlackBotToken)) {
            result.token = userSlackBotToken;
            result.message = 'Token fetched from input';
        }

        // Handle cases where no token is available
        if (String.isBlank(result.token)) {
            result.message = 'No token provided';
        }

        return result;
    }

    public static ReturnSlackCallout SlackApiCallout(String endpoint, String method,
```

```

String token, String payload) {
    ReturnSlackCallout result = new ReturnSlackCallout();

    // Check if the token is provided
    if (String.isBlank(token)) {
        result.message = 'No token provided';
        return result;
    }

    // Check if the endpoint is provided
    if (String.isBlank(endpoint)) {
        result.message = 'No endpoint provided';
        return result;
    }

    try {
        // Setup the HTTP request
        HttpRequest req = new HttpRequest();
        req.setEndpoint(endpoint);
        req.setMethod(method);
        req.setHeader('Content-Type', 'application/json');
        req.setHeader('Authorization', 'Bearer ' + token);
        if (!String.isBlank(payload)) {
            req.setBody(payload);
        }

        Http http = new Http();
        result.res = http.send(req);

        if (result.res.getStatusCode() == 200) {
            System.debug('Slack API callout successful: ' + result.res.getBody());
            result.message = 'Slack API callout successful';
            result.status = 'success';
        } else {
            System.debug('Slack API callout failed: ' + result.res.getBody());
            result.message = 'Slack API callout failed: ' + result.res.getBody();
            result.status = 'error';
        }
    } catch (Exception e) {
        System.debug('HTTP response error: ' + e.getMessage());
        result.message = 'HTTP response error: ' + e.getMessage();
        result.status = 'error';
    }

    return result;
}

public static List<GetUsersInfoSlack.UserInfoWrapper> getUsersInfo(
    List<Id> salesforceUserIds,
    List<String> slackIds
) {
    List<GetUsersInfoSlack.UserInfoWrapper> results = new

```

```

List<GetUsersInfoSlack.UserInfoWrapper>();

// If both lists are empty, fetch all Salesforce Users
if ((salesforceUserIds == null || salesforceUserIds.isEmpty()) && (slackIds ==
null || slackIds.isEmpty())) {
    salesforceUserIds = new List<Id>();
    for (User u : [SELECT Id FROM User WHERE IsActive = TRUE]) {
        salesforceUserIds.add(u.Id);
    }
}

// Get mappings from the Slack UserMappingService
String teamId = 'T08LMTRBD2B'; // More generic ??
Map<String, String> sfToSlackMap = new Map<String, String>();
if (salesforceUserIds != null && !salesforceUserIds.isEmpty()) {
    sfToSlackMap = Slack.UserMappingService.getSlackUserIdMap(salesforceUserIds,
teamId);
}

Map<String, String> slackToSfMap = new Map<String, String>();
if (slackIds != null && !slackIds.isEmpty()) {
    slackToSfMap = Slack.UserMappingService.getSalesforceUserIdMap(slackIds);
}

Set<String> allSfIds = new Set<String>();
if (sfToSlackMap != null && !sfToSlackMap.isEmpty()) {
    allSfIds.addAll(sfToSlackMap.keySet());
}
if (slackToSfMap != null && !slackToSfMap.isEmpty()) {
    allSfIds.addAll(slackToSfMap.values());
}

// Remove technical IDs from the set
allSfIds.remove('force__cornerstone');

// Query User information
Map<Id, User> userMap = new Map<Id, User>([SELECT Id, Name FROM User WHERE Id IN
:allSfIds]);

for (String sfId : allSfIds) {
    User user = userMap.get((Id) sfId); // Cast String to Id for querying User
    if (user == null) {
        continue;
    }

    String slackId = sfToSlackMap != null && sfToSlackMap.containsKey(sfId) ?
sfToSlackMap.get(sfId) : null;

    if (slackId == null && slackToSfMap != null && !slackToSfMap.isEmpty()) {
        for (String sId : slackToSfMap.keySet()) {
            if (slackToSfMap.get(sId) == sfId) {
                slackId = sId;
            }
        }
    }
}

```

```

        break;
    }
}

results.add(new GetUsersInfoSlack.UserInfoWrapper(user.Name, user.Id,
slackId));
}

return results;
}
}

```

Methods

getBotToken(agentName, userSlackBotToken)

Signature

```
public static ReturnToken getBotToken(String agentName, String userSlackBotToken)
```

Parameters

Name	Type	Description
agentName	String	
userSlackBotToken	String	

Return Type

ReturnToken

SlackApiCallout(endpoint, method, token, payload)

Signature

```
public static ReturnSlackCallout SlackApiCallout(String endpoint, String method,
String token, String payload)
```

Parameters

Name	Type	Description
endpoint	String	
method	String	
token	String	
payload	String	

Return Type

ReturnSlackCallout

getUsersInfo(salesforceUserIds, slackIds)

Signature

```
public static List<GetUsersInfoSlack.UserInfoWrapper> getUsersInfo(List<Id>
salesforceUserIds, List<String> slackIds)
```

Parameters

Name	Type	Description
salesforceUserIds	List<Id>	
slackIds	List<String>	

Return Type

List<GetUsersInfoSlack.UserInfoWrapper>

Classes

ReturnToken Class

Fields

token

SIGNATURE

```
public token
```

TYPE

String

message

SIGNATURE

```
public message
```

TYPE

String

ReturnSlackCallout Class

Fields

status

SIGNATURE

public status

TYPE

string

message

SIGNATURE

public message

TYPE

String

res

SIGNATURE

public res

TYPE

HttpResponse